
PART IV — FROM RAG TO AGENTIC RAG

Chapter 15

What “Agentic” Really Means

“Agency begins where a fixed pipeline runs out of judgment.”

Chapter 15 — What “Agentic” Really Means

The ingestion and retrieval chapters gave us a dependable RAG pipeline: turn a question into an embedding, retrieve nearby chunks, place those chunks beside the question, and ask an LLM to answer. That sequence is useful precisely because it is predictable. Yet predictability becomes a weakness when the first retrieval is incomplete, ambiguous, or simply aimed at the wrong interpretation of the question.

This chapter marks the conceptual transition from RAG as a fixed function to RAG as a controlled decision process. We will use Memora’s own evolution as evidence: the project began with `query.py`, a single-pass `embed → retrieve → answer` path, then introduced an agentic loop that can reformulate queries, accumulate evidence, evaluate quality, and stop when either the answer is good enough or a budget is exhausted. By the end, you will be able to explain agency without mysticism, identify the four decisions an agent makes, and decide when the extra complexity is justified.

Definition — Agentic RAG

A retrieval-augmented generation system in which a model participates in choosing the next operation from the current state—for example, whether to retrieve again, what query to issue, which knowledge source to use, or when to stop—while an orchestrator enforces safety, ordering, and resource limits.

15.1 The fixed pipeline limitation of traditional RAG

A traditional RAG pipeline is a straight line. The same stages run in the same order for every question, whether the question is trivial, ambiguous, multi-part, or impossible to answer from the corpus. A typical implementation embeds the user query, retrieves `top_k` chunks, formats a prompt, calls the model once, and returns the result. The model may reason inside the final call, but it cannot alter the surrounding program’s plan.

```
def answer_once(question, retriever, llm):
    chunks = retriever.retrieve(question, top_k=5)
    prompt = format_prompt(question, chunks)
    return llm.invoke(prompt)
```

The weakness is not that this function is badly written. Its weakness is that every important decision was made before runtime. The developer chose one query, one retriever, one value of `top_k`, and one generation attempt. If a user asks, “Compare the clinical and engineering meanings of ASD, then explain which documents support each,” the pipeline cannot notice that the acronym has two domains and issue separate searches. It simply searches the original sentence and hopes the nearest chunks are sufficient.

Analogy — A fixed train route

Traditional RAG is like boarding a train with one predetermined route. It is efficient when your destination lies on that line. If the first track is blocked or the destination requires a transfer, the train does not inspect the situation and redesign the journey—it continues along the timetable it was given.

Memora’s architecture ledger describes this original stage as baseline RAG: embed the query, retrieve once, build a prompt, and answer. It had no iteration, no self-correction, and no persistent run state. Three recurring failures motivated the transition: poor first retrieval could not be repaired, the same weak search could be repeated across sessions, and user feedback could not influence a later attempt.

- A vague question remains vague because no stage rewrites it.
- A partially relevant retrieval becomes the final context because no stage judges coverage.
- A multi-part question receives one undifferentiated search instead of targeted sub-queries.
- A confident answer can be returned even when the retrieved evidence is thin or contradictory.

Worked example — one acronym, two domains

Suppose the corpus contains clinical material about autism spectrum disorder and engineering manuals about adjustable speed drives. The user asks, “What causes ASD failures, and how should they be diagnosed?” A single-pass retriever embeds the whole sentence as one vector. The nearest results may cluster around whichever meaning dominates the embedding space, so the answer can silently mix medical diagnosis with equipment failure modes. Increasing ‘top_k’ does not solve the ambiguity; it merely retrieves more material under the same unresolved intent.

A fixed pipeline can handle this only if a developer anticipated the ambiguity and wrote a special-case disambiguator. An agentic pipeline can detect that retrieved chunks belong to incompatible domains, preserve both interpretations in state, and ask a clarification question or issue two labelled sub-queries. The crucial improvement is not “more reasoning.” It is the ability to change the next operation after observing evidence that the initial interpretation was unsafe.

This example also shows why agentic behavior must remain bounded. The agent may choose between clarification and two approved searches, but it should not invent a third knowledge source or continue searching indefinitely. The orchestrator records the interpretations tried, caps retrieval, and requires the final answer to state which meaning it addresses. Runtime judgment operates inside a deterministic envelope.

15.2 Autonomous agents making dynamic retrieval decisions

An agent changes the shape of the computation at runtime. It observes the current state, selects an allowed action, receives the action’s result, and then decides again. “Autonomous” does not mean unconstrained or conscious. It means that the next action is not completely hard-coded in advance. The model has bounded discretion inside a program designed by the developer.

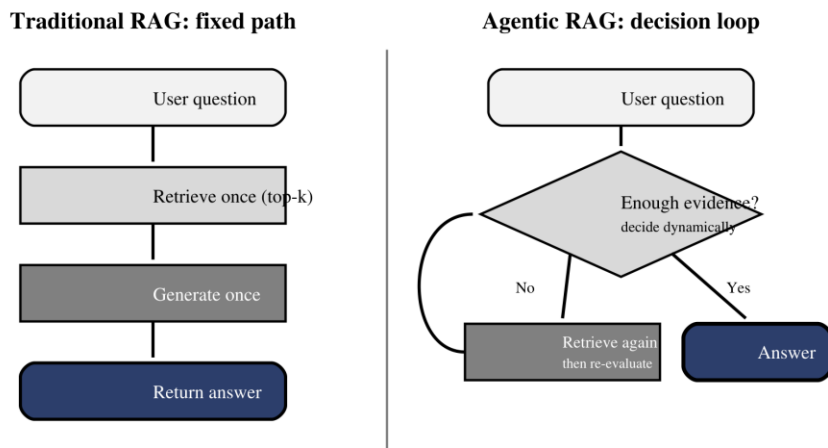


Figure 15.1 — A fixed RAG path compared with a bounded agentic decision loop.

The distinction in Figure 15.1 is control flow. Both sides may use the same embedding model, vector database, and generation model. On the left, those components are connected by a fixed sequence. On the right, evidence quality feeds back into the next decision. The agent may reformulate the query and retrieve again; the orchestrator may force a missing validation step; or the run may stop because a quality condition or budget has been reached.

Common pitfall — Calling any LLM workflow an agent

A pipeline does not become agentic merely because an LLM is present. If the LLM always receives one prompt and the application always performs the same next step, the workflow is still fixed. Agency requires a runtime choice that can change the subsequent path.

Memora demonstrates bounded autonomy clearly. The earlier imperative implementation exposed retrieval and quality-check tools to the LLM, but the outer loop enforced phase ordering and caps. The later LangGraph implementation moved more of that discipline into graph topology: nodes and conditional edges make valid transitions explicit. In both versions, the model proposes; the orchestrator permits, corrects, or stops.

Layer	Responsibility	Memora example
Model	Chooses or proposes the next semantic action	Generate query variants; judge whether evidence covers the question
Tool	Performs a bounded external operation	Retrieve from document and learned-QA collections
State	Carries run-specific evidence and progress	Variants tried, chunks, draft, verdicts, retry counters
Orchestrator	Enforces allowed order, budgets, and termination	Phase guards in the loop; nodes and routes in LangGraph

15.3 The four agent decisions: when, what, where, and how many times

The word “agent” becomes practical when we decompose it into four retrieval decisions. These decisions are related, but each solves a different failure mode. A well-designed system may give the model discretion over some of them while keeping the others deterministic.

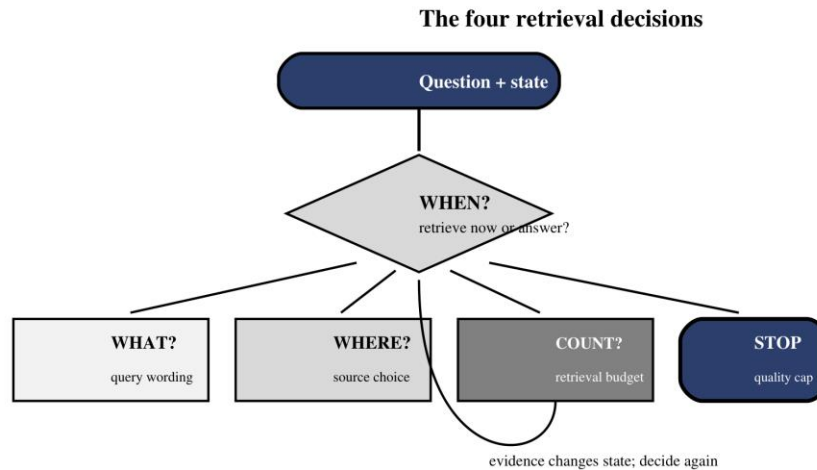


Figure 15.2 — Four bounded decisions turn retrieval into an adaptive process.

Figure 15.2 groups the agentic behavior into four decisions that can be tested independently: whether to retrieve, what to retrieve, when evidence is sufficient, and whether the draft passes a quality gate. The value lies in the controlled choices, not in making every step autonomous.

When to retrieve

The agent decides whether the current state already contains enough evidence to answer. It may retrieve immediately for a factual question, skip retrieval for a command such as ‘stats’, or retrieve again after a coverage judge identifies a missing sub-question. This decision prevents both under-retrieval and wasteful tool use.

What to retrieve

The literal user sentence is not always the best search query. The agent can resolve an acronym, isolate entities, split a comparison into sub-queries, or avoid variants that previously produced a bad answer. In Memora, query-variant generation turns one request into several search directions, while failed variants and thumbdown memory can block known bad trajectories.

Where to retrieve

A mature agent may have several knowledge sources: raw documents, learned Q&A memory, a relational database, a web search tool, or an internal API. Memora keeps source documents and learned Q&A as independent retrieval tracks because their evidence has different provenance and thresholds. The decision is not merely “search or do not search,” but “which source is appropriate for this need?”

How many times to retrieve

Repeated retrieval can improve coverage, but an unbounded loop can become slower, more expensive, and less reliable with every turn. The imperative Memora loop therefore used iteration and retrieval caps; the

LangGraph rewrite bounds retries structurally through routes and retry limits. The guiding principle is simple: continue only while the next retrieval has a plausible chance of adding missing evidence.

```
while not state.done:
    action = decide_next_action(state)
    if action.kind == "retrieve" and state.within_budget():
        evidence = retrieve(action.query, action.source)
        state = evaluate_and_accumulate(state, evidence)
    else:
        state = draft_validate_or_stop(state)
```

Reading an agent trace as a sequence of decisions

An agent trace should make each decision inspectable. Begin with the user question and the state before the action. Record the chosen action, its arguments, the evidence returned, the state changes, and the reason for continuing or stopping. Without this structure, a trace is merely a long transcript; with it, the trace becomes an explanation of control flow.

Turn	Observed state	Decision and reason
1	No evidence; ambiguous acronym	Retrieve two domain-specific variants to establish meaning
2	Clinical and engineering chunks both present	Ask for clarification rather than merge incompatible domains
3	User selects engineering meaning	Retrieve drive-failure and diagnostic-code evidence
4	Coverage judge finds causes and tests	Stop retrieval and synthesize a grounded answer

The reason column is especially important during debugging. If a retrieval did not improve coverage, the next question is whether the model made a poor semantic choice, the retriever returned weak evidence, or the orchestrator permitted an invalid transition. Separating those causes prevents prompt tuning from becoming the default response to every failure.

15.4 Agentic RAG versus traditional RAG — a visual comparison

The two designs are not enemies. Traditional RAG is a valuable baseline and often the correct production choice. Agentic RAG adds adaptive control flow on top of the same retrieval fundamentals. The table below compares the engineering consequences rather than treating “agentic” as an automatic upgrade.

Dimension	Traditional RAG	Agentic RAG
Control flow	Predetermined sequence	Conditional path chosen from current state
Retrieval	Usually, one query and one pass	May reformulate, branch, validate, and repeat
State	Often only question + retrieved chunks	Tracks evidence, attempts, verdicts, and budgets
Failure handling	Fallbacks written around the pipeline	Can select a corrective action, within enforced limits
Latency and cost	Lower and predictable	Higher and variable; must be budgeted
Debugging	Simple linear trace	Requires node/action traces and termination diagnostics
Best fit	Stable, well-scoped questions	Ambiguous, multi-step, or evidence-sensitive questions

Notice that “better” depends on the workload. If one retrieval reliably answers the question, adding an agent creates more places to fail. If the workload routinely requires clarification, query decomposition, cross-source evidence, or self-correction, a fixed pipeline forces the developer to anticipate every branch manually. Agency earns its cost when runtime judgment solves a real variation in the task.

15.5 When you actually need an agent — and when you do not

Use the smallest control system that reliably solves the problem. Begin with a fixed RAG baseline, instrument it, and study the failures. Move toward an agent only when the evidence shows that a fixed path cannot handle meaningful variation without becoming a maze of special cases.

Signal	Prefer fixed RAG	Prefer agentic RAG
Question shape	Short, repetitive, single intent	Ambiguous, multi-part, or investigative
Evidence	One corpus; consistent retrieval quality	Multiple sources or uncertain coverage
Corrective action	Retrying the same query is sufficient	Must reformulate, branch, or validate
Operational constraints	Strict latency and predictable cost	Extra latency is acceptable for better evidence
Risk	Low-stakes answer or human review	High value from explicit grounding and quality gates

Analogy — Cruise control versus a driver

A fixed RAG pipeline resembles cruise control: excellent on a clear road with a stable destination. An agent resembles a driver who can change lanes, slow down, or take a detour. You do not hire a driver merely to hold a constant speed; you need one when the road demands judgment.

A practical decision test is to ask three questions. First, can you state the complete workflow before the request arrives? Second, do failures require a different next action rather than merely a retry? Third, can you define hard limits that keep the adaptive loop safe? If the answers are “yes, no, and not applicable,” keep the fixed pipeline. If they are “not always, yes, and yes,” an agentic design is justified.

A staged migration instead of an agent rewrite

Do not replace a working fixed pipeline with a fully autonomous loop in one step. First, preserve the single-pass implementation as a baseline. Add retrieval logging and a coverage evaluator. Next, introduce one bounded corrective action—usually query reformulation after an insufficient verdict. Only then consider multiple sources, branching, persistent failure memory, or framework-level orchestration. Each stage should demonstrate a measurable improvement on a fixed evaluation set before the next source of complexity is added.

- Baseline: one retrieval, one answer, complete latency and quality measurements.
- Gate: evaluate evidence coverage before generation and record the failure reason.
- Single correction: permit one reformulated retrieval when coverage is insufficient.
- State: retain tried variants and validated evidence so retries add information.
- Budgets: enforce hard limits before increasing the available action set.
- Expansion: add another tool or source only for a demonstrated failure category.

This staged approach makes “agentic” an evidence-based architectural decision rather than a branding choice. It also produces clean ablation points: you can compare the baseline, the quality gate, and the corrective loop under identical questions. If the agentic layer adds latency without improving the failure cases it was introduced to solve, the correct engineering response is to simplify it.
